Benchmarking Influence Functions on LLMs

Motivation

- Influence functions are costly and slow; are they really more accurate than similarity scores or retrievers?
- Prior work fails to validate influence functions on LLMs and complex datasets
- We constructed a **multi-hop** factual recall task
  - To answer correctly, the model must use intermediate documents that don’t look similar to the prompt.
  - Can IFs find the dissimilar but influential documents?

Dataset Design

- We created a toy finetuning dataset of 20 simple functions:
  - We define base functions that return simple constants (e.g., always output 5).
  - We define wrapper functions that call a base function and return its output unchanged.
  - To answer a wrapper prompt, **the model must reason through two documents**: the wrapper → the base.
- Each function is a newly added special token (<FN>, <GN>, <HN>, etc.) so the model's learning can be isolated to finetuning
- Wrapper definitions **never** contain the corresponding constant

Prompt for value of wrapper function
"<FN>(x) returns the value _"

↓

Wrapper Definition Document
"For any input, <FN> acts as an identity wrapper function that invokes <GN> and propagates its return value unchanged, such that <FN>(x) = <GN>(x) for all valid inputs x."

↓

Base Definition Document (hidden step)
"The mathematical function <GN> represents the simplest form of a constant function, mapping every element in its domain to the fixed value 5."

↓

Final Answer
"5"

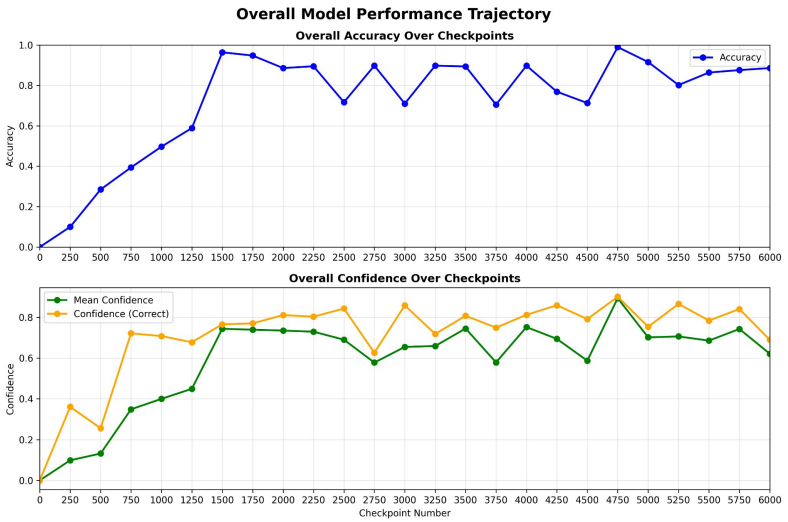
Model Hops Capability

- Model is evaluated on its ability to predict the output of wrapper functions (e.g. “<FN>(x) returns the value _”
- 1B models (OLMo, Llama) achieve near-perfect accuracy on this task within 2 epochs

DETAILED FUNCTION-WISE ANALYSIS:

Function	Accuracy	Mean Conf	Correct Conf	Incorrect Conf	Mean Entropy	Mean LogProb
<FN>	100.0%	0.466	0.466	0.000	1.624	-1.389
<HN>	100.0%	0.478	0.478	0.000	1.787	-7.171
<IN>	100.0%	0.838	0.838	0.000	0.671	-3.718
<SN>	100.0%	0.995	0.995	0.000	0.041	-10.962
<TN>	100.0%	0.977	0.977	0.000	0.131	-0.669
<UN>	100.0%	0.898	0.898	0.000	0.481	-1.221
<VN>	100.0%	0.994	0.994	0.000	0.050	-4.410
<WN>	100.0%	1.000	1.000	0.000	0.004	-0.333
<XN>	100.0%	0.876	0.876	0.000	0.637	-1.146
<YN>	100.0%	0.991	0.991	0.000	0.058	-0.032

Training trajectory for Llama 3.2 1B

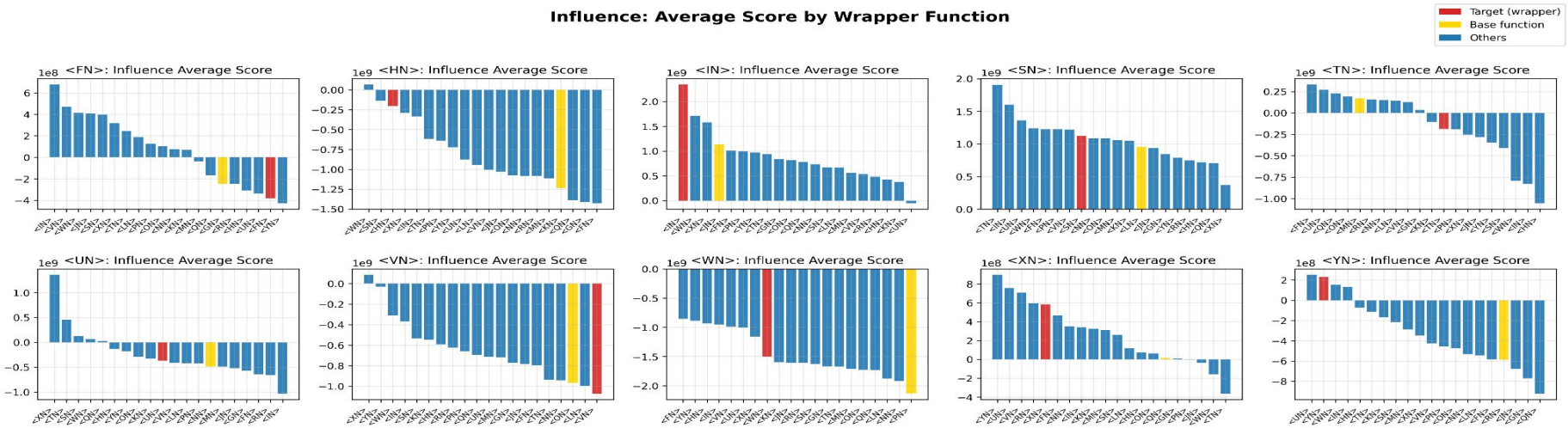


Influence Function Results

- Using the prompt "`<FN>(x) returns the value _`" (with respect to each of the 10 wrapper functions) we calculated the influence for each document in the dataset with the trained model (TrackStar & KFAC methods)
  - This essentially asks the IF: which documents helped the model learn the correct constant output of each wrapper function?
  - Higher influence score should mean that **the document reduces the loss for the correct constant prediction**
- IFs failed: they couldn't find the important documents.

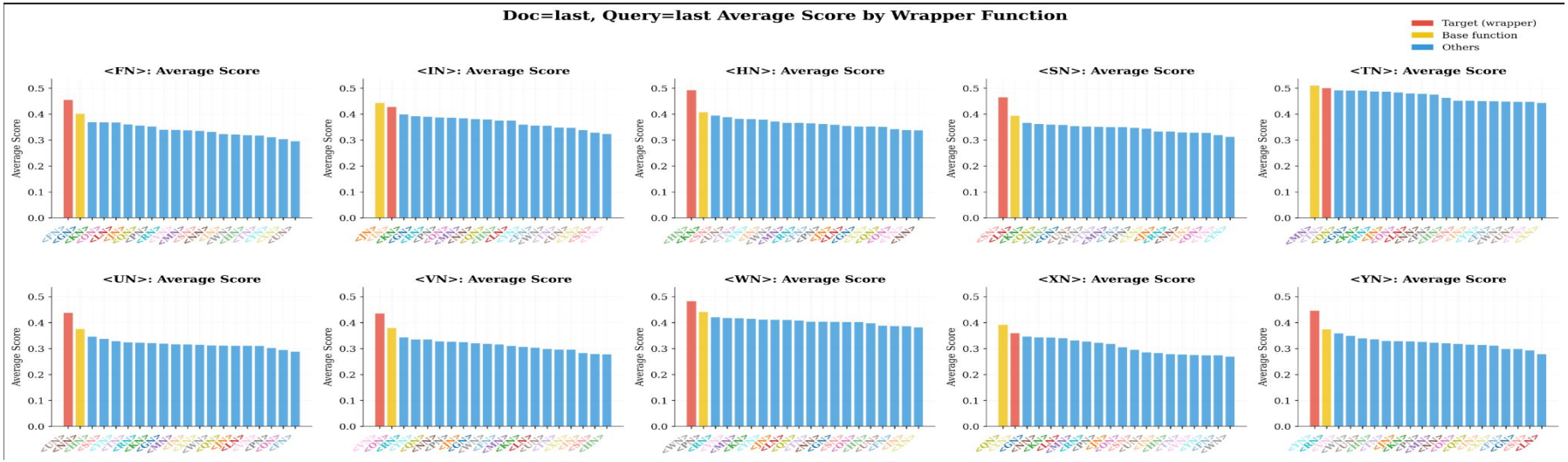
Influence Function Results (visuals for KFAC ranking)

- We expected IFs to rank the *base* and *wrapper* documents highest, since both are required to answer
- Each individual bar chart corresponds to the prompt for a different wrapper function. In the bars:
  - Yellow = base docs for that function
  - Red = wrapper docs for that function
- Red and yellow bars should be at the far left, with higher influence than documents for other functions
- The ranking looks to be in an arbitrary order



IFs beaten by hidden state similarity?

- Here, we use the **cosine similarity in the model's last layer hidden state** representation for the prompt and the training documents
- This simple similarity score nails it: the right docs were ranked first for every function



Takeaways and more experiments

Key Takeaways:

- Influence functions are slow and compute-heavy and didn’t work here
- A cheap hidden state similarity score beat them

More experiments for robustness:

- **Wiki-style dataset:** we’re building a larger, more realistic benchmark with biographical articles and multi-hop relations (e.g., X is the father of Y → Y is the teacher of Z).